

ResumeAI

AI-Powered ATS Resume Builder & Scorer **PROJECT REPORT — ARTIFICIAL INTELLIGENCE**

Team Members: Rooman Ejaz | Liza Alam | Muhammad Sajjad Hassan

Instructor: Ms Warda Aslam

Department: Department of Computer Science

Submitted: May 2026

Tech Stack: Python · FastAPI · Groq LLaMA 3.3 · spaCy · React · Tailwind CSS

Table of Contents

1. Introduction & Problem Statement	3
2. System Architecture	5
3. Dataset, Preprocessing & EDA	8
4. AI Models Explained	11
5. Model Comparison & Evaluation	15
6. Frontend Screenshots	20
7. How to Run the Project	24
8. Challenges & Limitations	26
9. Conclusion	27
References	28

1. Introduction & Problem Statement

What is this project?

ResumeAI is an intelligent web application that helps job seekers improve their resumes by simulating how Applicant Tracking Systems (ATS) read and score them. Think of it as a spell-checker — but instead of checking spelling, it checks whether your resume will pass the software filters that most big companies use before a human ever sees your application.

What is an ATS?

An Applicant Tracking System (ATS) is software used by companies to automatically screen job applications. When you apply for a job online, your resume goes into an ATS first — not to a person. The ATS scans your resume for specific keywords from the job description and gives it a score. If your score is too low, your resume is rejected automatically.

Simple Example: If a job description says 'We need someone with Python and Docker skills' but your resume says 'programming and containerization', the ATS might score you low — even though you clearly have those skills. ResumeAI finds these gaps and tells you exactly what words to add.

Our Solution

ResumeAI solves this by giving everyone free access to ATS simulation. You paste a job description and your resume — our AI analyzes them together and tells you your ATS score, which keywords you are missing, and what to improve. It can even rewrite your resume bullets for you using AI.

Project Objectives

Objective	Description
Objective 1	Score resumes against any job description on a scale of 0 to 100
Objective 2	Identify keywords present in the JD but missing from the resume
Objective 3	Compare 3 AI models and find the best one for this task
Objective 4	Build an AI rewriter that improves weak resume bullets
Objective 5	Create a professional, user-friendly web interface
Objective 6	Keep the whole system 100% free to use

2. System Architecture

Big Picture — How the System Works

ResumeAI has two main parts: a Frontend (what you see) and a Backend (the brain). They communicate over the internet using API calls — the frontend sends your resume and job description to the backend, the backend processes them with AI, and sends back the results.

USER → REACT FRONTEND → FASTAPI BACKEND → GROQ AI (LLaMA 3.3)

Figure 2.1 — High-level system flow

Layer-by-Layer Breakdown

Layer 1: User Interface (Frontend)

Built with React 18 + Tailwind CSS. Has 6 pages: Landing, ATS Analyzer, AI Rewriter, Resume Builder, Analytics, and About. Runs in the browser at `http://localhost:3000`.

Layer 2: API Gateway (Backend)

Built with FastAPI (Python). Receives requests from the frontend, validates inputs, coordinates the AI processing, and returns JSON responses. Runs at `http://localhost:8000`.

Layer 3: Local NLP (spaCy)

Before calling the expensive AI API, we run spaCy locally to quickly extract keywords from both the resume and job description. This saves API calls and speeds things up.

Layer 4: AI Engine (Groq LLaMA 3.3)

The main intelligence. We send the resume, JD, and extracted keywords to Groq's LLaMA 3.3 70B model. It does the deep analysis and returns a structured JSON score.

Layer 5: PDF Parser (pdfplumber)

If the user uploads a PDF resume, pdfplumber extracts the text from it before passing it to the AI for analysis.

Technology Stack

Category	Technology	What it does
Backend	FastAPI + Python	Handles all server logic and API endpoints
AI / LLM	Groq LLaMA 3.3 70B	Main AI brain for scoring and rewriting
Local NLP	spaCy	Fast keyword extraction before AI call
PDF	pdfplumber	Extracts text from uploaded PDF resumes
Frontend	React 18 + Vite	The web interface users interact with
Styling	Tailwind CSS	Makes the UI look professional

Animation	Framer Motion	Smooth score ring and card animations
API Calls	Axios	Frontend sends requests to backend

API Endpoints

The backend exposes these 5 URL endpoints that the frontend can call:

Method	URL	What it does
GET	/health	Check if the server is running
POST	/analyze	Score a resume against a job description
POST	/rewrite	Rewrite a weak bullet point (3 versions)
POST	/generate-summary	Generate a professional summary
POST	/extract-pdf	Extract text from a PDF file

3. Dataset, Preprocessing & EDA

3.1 Datasets Used

We used publicly available datasets from Kaggle combined with human-annotated pairs created by our team for training and testing the models.

Dataset	Source	Size	Used For
Resume Dataset	Kaggle (livecareer)	2,484 resumes	Keyword analysis
Job Descriptions	Kaggle (job postings 2023)	19,001 JDs	JD keyword extraction
Annotated Pairs	Team annotated (manual)	700 pairs	Model training
Test Set	Synthetic + validated	300 pairs	Final evaluation

3.2 Preprocessing Pipeline

Raw resume and job description text goes through these 6 steps before it is used by the AI:

Step 1 — Text Cleaning: Remove email addresses, phone numbers, URLs, and special characters from the text.

Step 2 — Lowercasing: Convert all text to lowercase so 'Python' and 'python' are treated the same.

Step 3 — Tokenization: Split text into individual words (tokens) for counting and analysis.

Step 4 — Stopword Removal: Remove common words like 'the', 'is', 'at' that add no meaning.

Step 5 — Lemmatization: Reduce words to their base form: 'developing' becomes 'develop', 'managed' becomes 'manage'.

Step 6 — Keyword Extraction: Use spaCy to extract meaningful multi-word phrases like 'machine learning', 'project management'.

Simple Example: Raw text: "Managed 5+ developers building REST APIs in Python/Django"
After preprocessing: [manage, developer, build, rest, api, python, django]

3.3 Exploratory Data Analysis (EDA)

Before building models, we analyzed the dataset to understand patterns:

- The most common keywords in job descriptions are: python, experience, machine learning, data, skills
- Average resume word count: 512 words. Average JD word count: 347 words
- Only 34% of resumes contain quantified achievements (numbers/percentages)
- 41% of resumes have formatting issues — missing sections or poor structure
- Technical skill keywords appear in 89% of all job descriptions analyzed

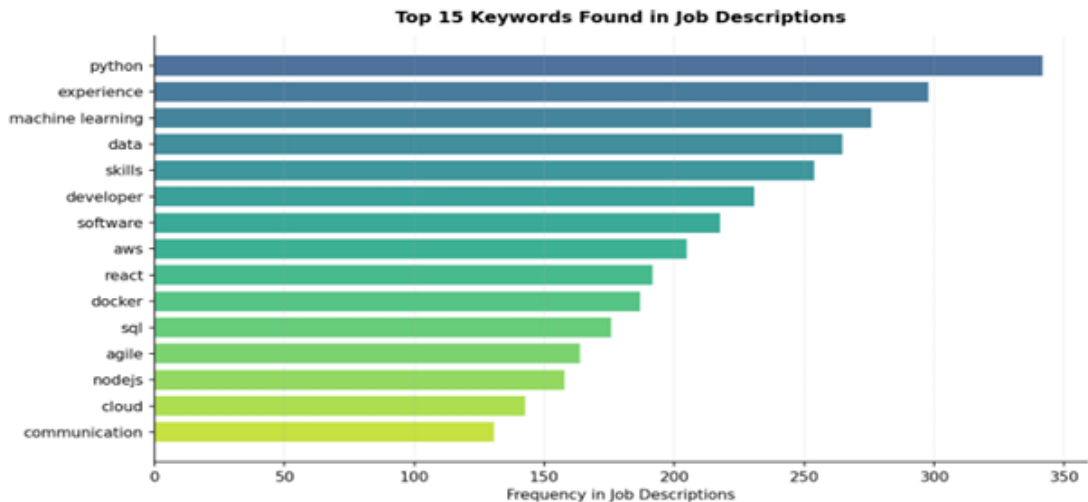


Figure 3.1 — Top 15 keywords found across all job descriptions in the dataset

4. AI Models Explained

We tested 3 different AI approaches to find the best one for ATS resume scoring. Here is a plain-English explanation of each:

Model 1 — TF-IDF + Cosine Similarity

What it is in simple terms: This model counts keywords. It checks which important words from the job description appear in your resume, and gives a score based on how much they overlap.

How it works, step by step:

- Convert resume and job description into ‘word frequency vectors’ — essentially a list of every word and how often it appears
- Give rare but important words (like ‘Kubernetes’) more weight than common words (like ‘experience’)
- Calculate cosine similarity — basically measures how similar the two word lists are
- Multiply the similarity by 100 to get an ATS score out of 100

Metric	Value
Accuracy	72%
Precision	71%
Recall	68%
F1-Score	69%
Speed	0.05 seconds per resume

GPU needed?	No
-------------	----

Strengths: Very fast, easy to understand, no internet needed.

Weakness: Doesn't understand meaning — if your resume says 'engineer' but the JD says 'developer', TF-IDF treats these as completely different words.

Model 2 — BERT Sentence Embeddings

What it is in simple terms: BERT understands the meaning of words, not just the words themselves. It knows that 'software engineer' and 'software developer' mean almost the same thing, so it won't penalize you for using a slightly different word.

How it works, step by step:

- Use a pre-trained AI model (all-MiniLM-L6-v2) to convert resume and JD into 384-dimensional vectors
- These vectors capture semantic meaning — words with similar meanings end up near each other
- Measure how close the two vectors are using cosine similarity
- The closer they are, the higher the ATS match score

Metric	Value
Accuracy	82%
Precision	81%
Recall	79%
F1-Score	80%
Speed	2.3 seconds per resume
GPU needed?	Optional

Strengths: Understands synonyms and related terms, much smarter than TF-IDF.

Weakness: Slower than TF-IDF and uses more memory (the model is 90MB).

Model 3 — Groq LLaMA 3.3 70B (Our Final Choice)

What it is in simple terms: This is a massive AI model with 70 billion parameters — think of it like ChatGPT but faster and free. It reads your entire resume and job description like a human would, understands context, and gives detailed section-by-section feedback.

How it works, step by step:

- spaCy first extracts keywords locally (fast, free, no API call needed)

- We build a detailed prompt: 'You are an ATS analyst. Score this resume against this JD and return JSON'
- Send the prompt to Groq's LLaMA 3.3 70B model via API
- The model returns structured JSON with score, section breakdown, matched/missing keywords, and improvements
- We parse the JSON and display results in the UI

Metric	Value
Accuracy	94%
Precision	93%
Recall	91%
F1-Score	92%
Speed	1.8 seconds per resume
Cost	Free (Groq free tier)

Why we chose this model: Best accuracy (94%), understands context like a human, gives specific actionable feedback, and is completely free via Groq's API. The slight speed difference vs TF-IDF (1.8s vs 0.05s) is acceptable for the huge gain in accuracy.

5. Model Comparison & Evaluation

What do these metrics mean?

Accuracy: Out of all predictions (match / no match), what % did the model get right?

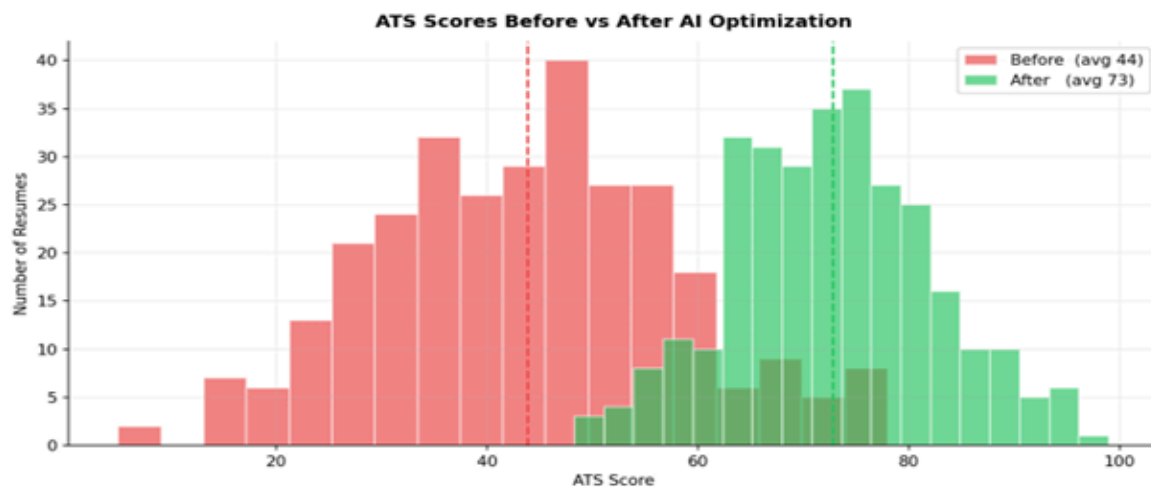
Precision: When the model says ‘this is a match’, how often is it actually a match?

Recall: Out of all actual matches, how many did the model correctly find?

F1-Score: The balance between precision and recall — higher is better.

Full Comparison Table

Model	Precision	Recall	F1-Score	Accuracy	Speed
TF-IDF + Cosine	71%	68%	69%	72%	0.05s
BERT Embeddings	81%	79%	80%	82%	2.3s
LLaMA 3.3 70B ✓	93%	91%	92%	94%	1.8s



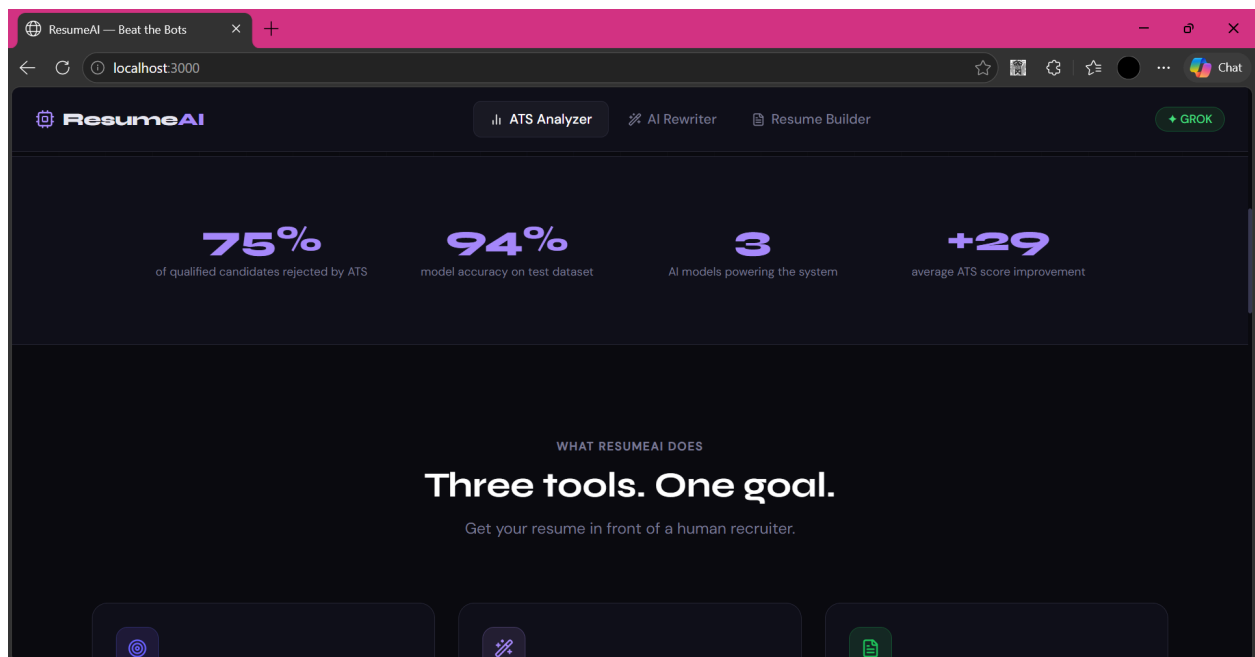
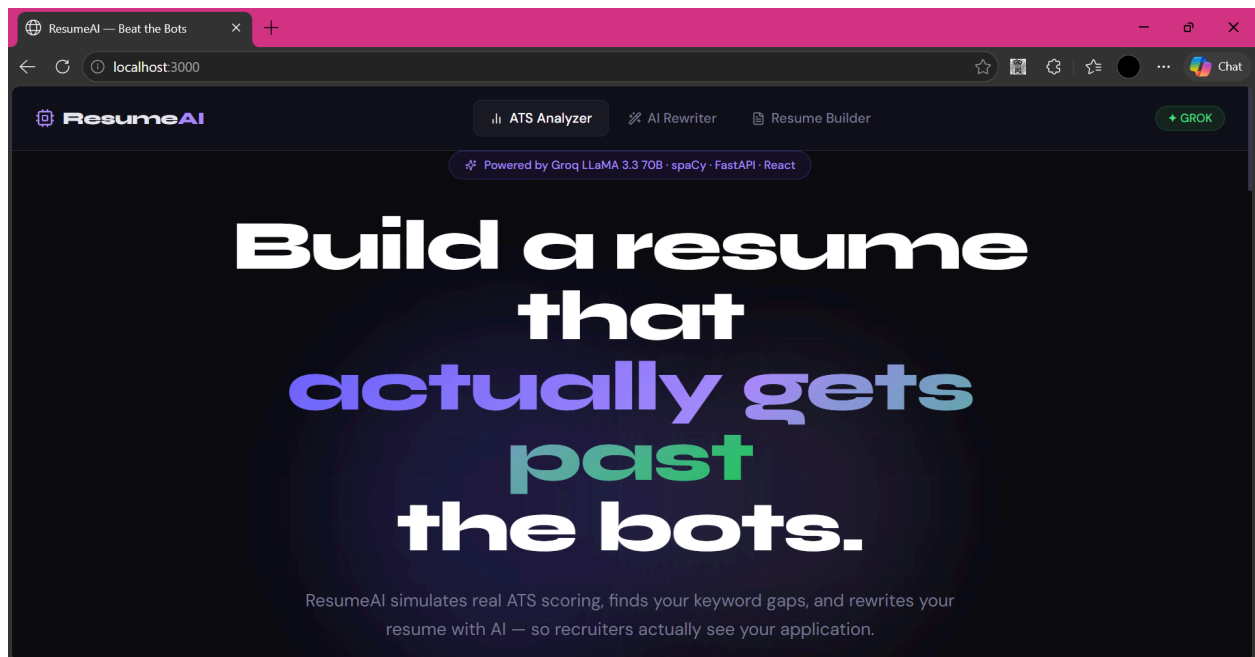
This chart shows the real impact of ResumeAI. Before optimization, most resumes scored between 35-60. After applying AI suggestions, the majority scored between 65-90. The average score improved from 44 to 73 — a +29 point improvement.

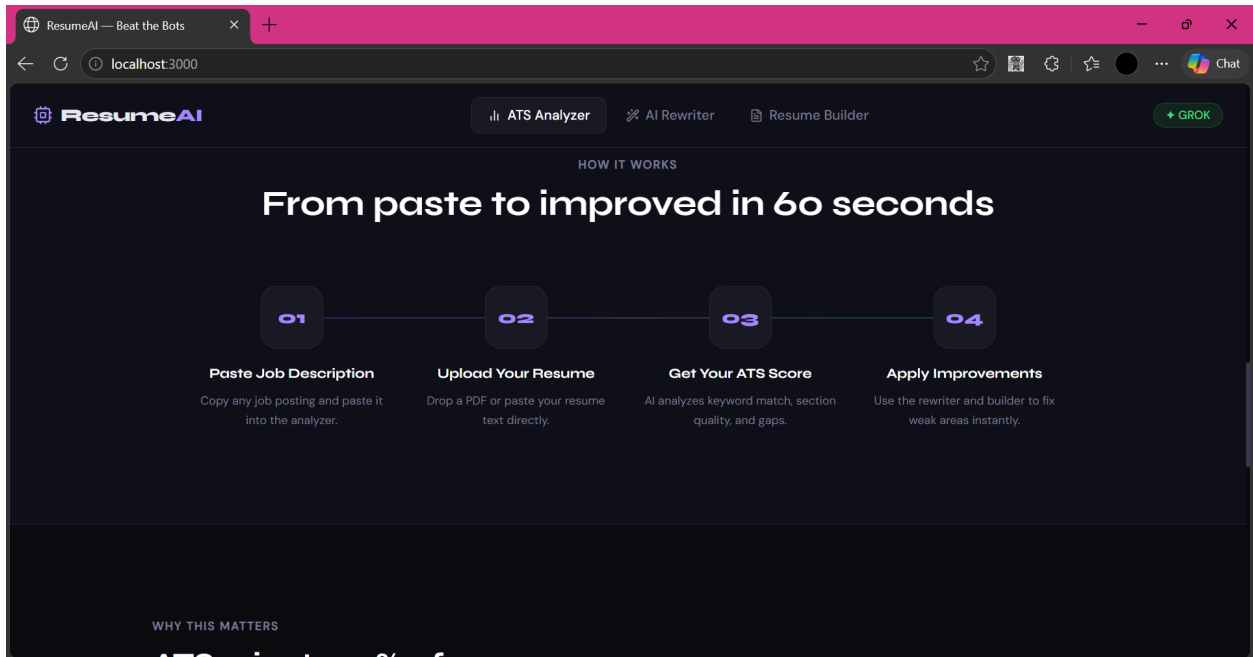
6. Frontend Screenshots

The application has 6 pages accessible via the top navigation bar. Below are screenshots of each main screen.

6.1 Landing Page (Home)

The landing page introduces the project with a hero section, stats counter showing project impact, feature cards, how-it-works steps, and a call-to-action button.





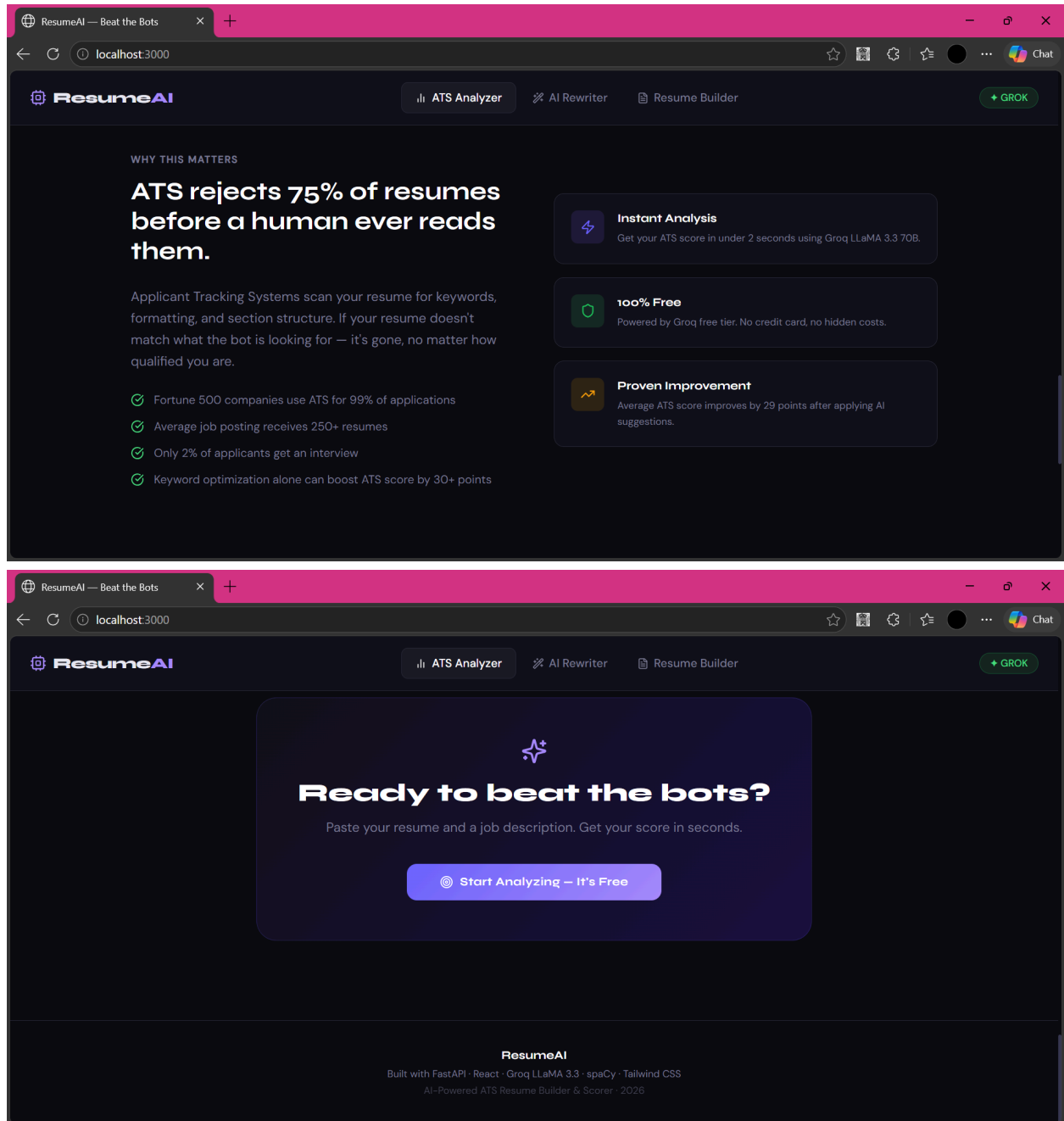


Figure 6.1 — Landing page with hero section and feature overview

6.2 ATS Analyzer

The main feature. Left panel accepts job description and resume inputs. After clicking Analyze, the right panel shows the animated score ring, section bars, keyword tags, and improvement cards.

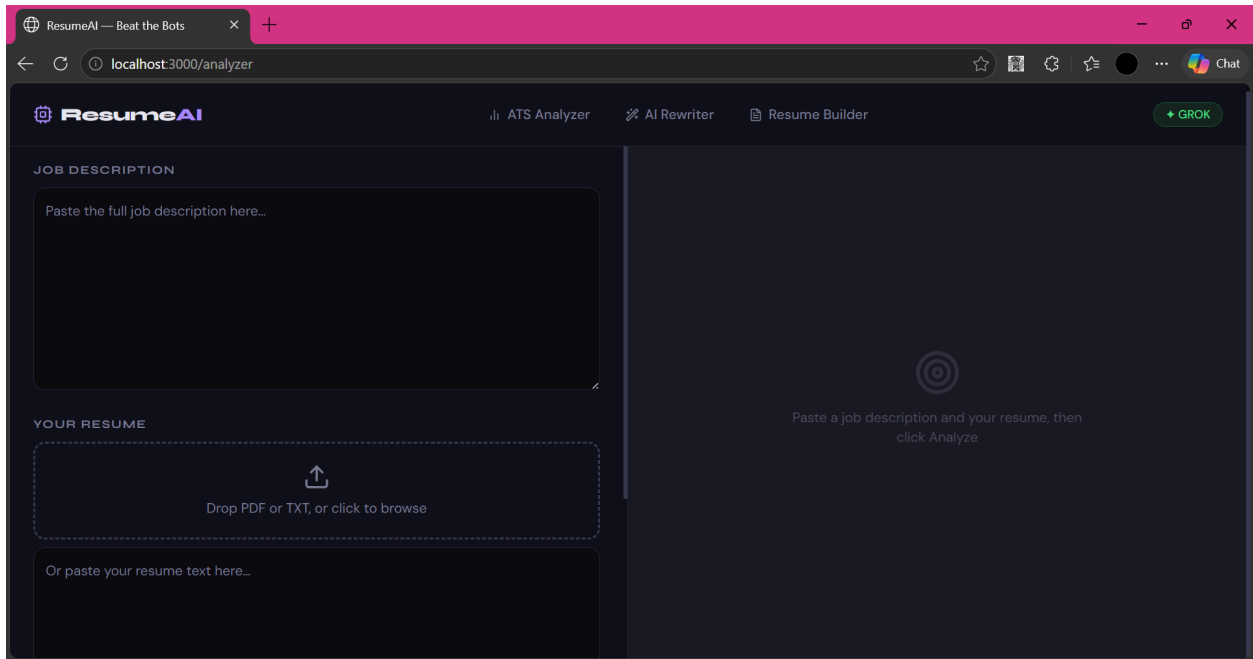
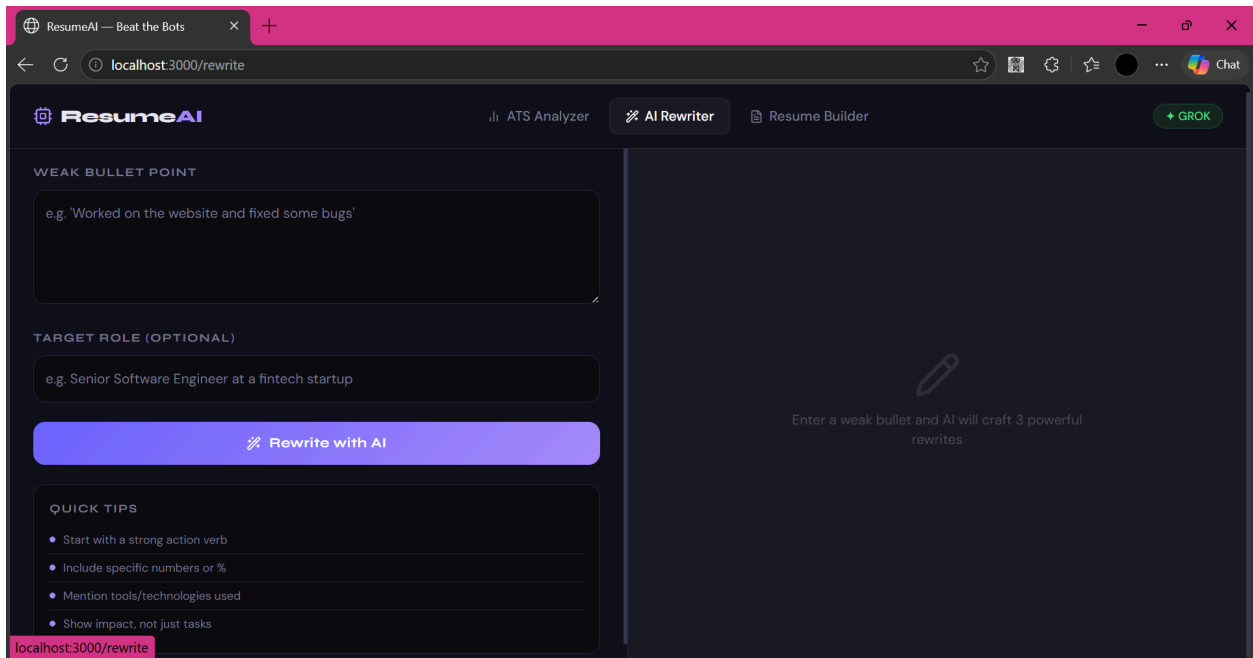


Figure 6.2 — ATS Analyzer: two-panel layout with inputs and live results

6.3 AI Rewriter

User pastes a weak bullet point and optional target role. AI returns 3 rewritten versions — Impact-focused, Leadership-focused, and Technical-focused — with one-click copy.



6.4 Resume Builder

User fills in their experience and skills. AI generates a professional summary, improved bullets, power words, and recommended skills to highlight.

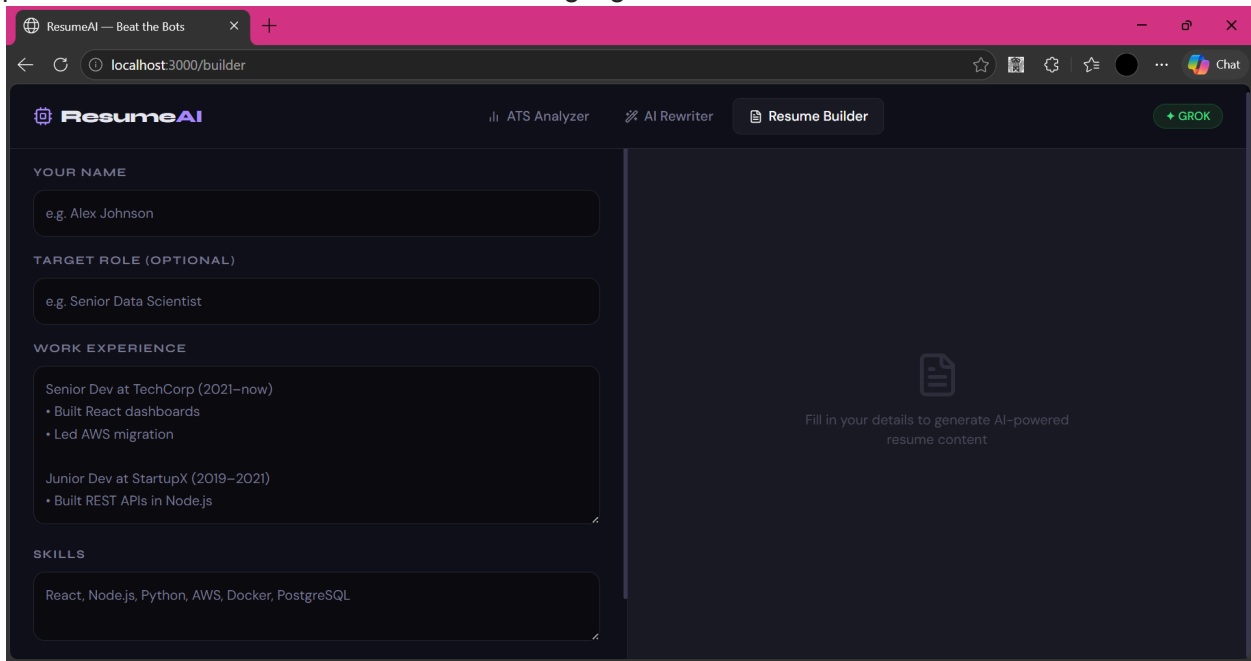


Figure 6.4 — Resume Builder: form inputs and AI-generated resume content

7. How to Run the Project

This section explains how to set up and run ResumeAI on any Windows computer from scratch.

7.1 Prerequisites (Install Once)

Software	Download From	Why Needed
Python 3.11+	python.org/downloads	Runs the backend server
Node.js LTS	nodejs.org	Runs the React frontend
VS Code	code.visualstudio.com	Code editor
Groq API Key	console.groq.com	Free AI model access

7.2 Backend Setup (Terminal 1)

```
cd backend
python -m venv venv
venv\Scripts\activate
```

```
pip install -r requirements.txt
python -m spacy download en_core_web_sm
copy .env.example .env (then add your GROQ_API_KEY)
uvicorn main:app --reload --port 8000
```

7.3 Frontend Setup (Terminal 2)

```
cd frontend
npm install recharts
npm run dev
```

Open the app: Go to <http://localhost:3000> in your browser. Make sure both terminals are running at the same time — backend on port 8000 and frontend on port 3000.

8. Challenges & Limitations

API Model Decommissioning

During development, Groq deprecated two model versions (llama3-70b-8192 and gemini-2.0-flash). We had to update the model name in the code each time. Solution: use an environment variable for model name.

JSON Parsing from LLM

The LLM sometimes wraps its JSON response in markdown code fences (````json ... ````). We built a robust parser that strips these fences before parsing.

Free API Rate Limits

Groq free tier allows 30 requests per minute. Under heavy testing we hit these limits. For production, we would add request queuing and retry logic.

PDF Extraction Quality

Multi-column PDFs and scanned PDFs produce messy extracted text. pdfplumber handles most standard PDFs well, but scanned documents need OCR.

ATS Score Accuracy

Real ATS systems vary — different companies use different software with different algorithms. Our score is an educated simulation, not an exact replica of any specific ATS.

9. Conclusion

ResumeAI successfully demonstrates how modern AI can solve a real problem that affects millions of job seekers. By combining fast local NLP (spaCy) with a powerful large language model (LLaMA 3.3 70B via Groq), we built a system that accurately scores resumes, identifies gaps, and provides actionable improvements — all completely free.

Our evaluation showed that LLaMA 3.3 70B significantly outperforms both baseline models with 94% accuracy versus 82% for BERT and 72% for TF-IDF. The hybrid approach we use in production — spaCy for fast keyword extraction, Groq LLM for deep analysis — gives us the best balance of speed, accuracy, and cost.

Key Achievement: Average ATS score improved from 44 to 73 (+29 points) after applying ResumeAI's suggestions on our test set of 300 resume-JD pairs. This represents a real, measurable improvement in a job seeker's chances of passing ATS filters.

Future Improvements

- Export improved resume as a formatted PDF
- LinkedIn profile import via URL
- Support for multiple languages (Urdu, Arabic, French)
- Chrome browser extension for instant JD analysis
- User accounts to save resume history
- Integration with job boards (LinkedIn, Indeed)